TypeScript : du code source au JavaScript exécuté !

I. Ce que TypeScript vérifie

Trois moments à distinguer

- **Compile time** : les types sont vérifiés avant l'exécution.
- **Build time** : les fichiers du projet sont assemblés et transformés.
- **Run time** : le JavaScript s'exécute dans le navigateur ou Node.js.

```
fichier .ts → vérification des types → JavaScript → exécution
```

Selon la configuration, la vérification des types et la transpilation peuvent utiliser deux outils distincts.

Documentation TypeScript - The Basics

Le contrôle de type a lieu avant l'exécution

```ts
interface Resource {
  id: number;
  name: string;
}

const resource: Resource = { id: 132, name: "ditto" };
console.log(resource.name);
```

> *Que se passe-t-il si je remplace `"ditto"` par `2` ?*

Documentation TypeScript - The Basics

Une donnée externe doit être typée `unknown`

```ts
async function loadPokemon(): Promise<void> {
  const response = await fetch("https://pokeapi.co/api/v2/pokemon/ditto");
  const payload: unknown = await response.json();
  console.log(payload);
}

await loadPokemon();
```

Essayez `console.log(payload.name)` dans la fonction : TypeScript refuse cet accès. PokéAPI renvoie un nom, mais le compilateur ne peut pas vérifier le JSON reçu.

Documentation TypeScript - unknown · Documentation PokéAPI - Pokémon

Une assertion ne valide pas le JSON

```ts
interface Resource {
  id: string;
  name: string;
}

async function loadPokemon(): Promise<void> {
  const response = await fetch("https://pokeapi.co/api/v2/pokemon/ditto");
  const payload = (await response.json()) as Resource;
  console.log(payload.id, typeof payload.id, payload.name);
}

await loadPokemon();
```

`id` est déclaré `string`, mais PokéAPI renvoie un nombre. `as Resource` ne vérifie rien à l'exécution.

Documentation TypeScript - Type assertions · Documentation PokéAPI - Pokémon

II. Valider un Pokémon avec Zod

Déclarer et utiliser un schéma

```ts
import { z } from "zod";

const pokemonSchema = z.object({
  id: z.number(),
  name: z.string().min(1)
});

async function loadPokemon(): Promise<void> {
  const response = await fetch("https://pokeapi.co/api/v2/pokemon/ditto");
  const payload: unknown = await response.json();
  console.log(pokemonSchema.parse(payload));
}

await loadPokemon();
```

Après `npm install zod`, `parse` renvoie le Pokémon validé ou lève une `ZodError`.

Documentation Zod - Basic usage · Documentation PokéAPI - Pokémon

Inférer le type du schéma

```ts
import { z } from "zod";

const pokemonSchema = z.object({
  id: z.number(),
  name: z.string().min(1),
});

type Pokemon = z.infer<typeof pokemonSchema>;

/* Équivalent à :
type Pokemon = {
  id: number;
  name: string;
};
*/

const pokemon: Pokemon = { id: 132, name: "ditto" };
console.log(pokemon.name);
```

Le type vient du schéma. Remplacez `name` par `42` pour observer l'erreur TypeScript. Remettez `name` en texte avant d'exécuter l'extrait.

Documentation Zod - Inferring types

Lire une erreur sans interrompre le programme

```
import { z } from "zod";

const pokemonSchema = z.object({ id: z.number(), name: z.string().min(1) });

const result = pokemonSchema.safeParse({ id: "132", name: "ditto" });

if (result.success) {
  console.log(result.data);
} else {
  console.log(`Problème sur la définition de ${result.error.issues[0]?.path.join(".")}`);
}
```

`safeParse` distingue succès et échec. Ici, le chemin affiché est `id`.

> *Comment corriger cette erreur ?*

Documentation Zod - Handling errors

Préparer la base du portfolio

Avant d'aller plus loin, on a besoin d'une base de données et d'une API REST.

Ouvrir le cours : construire la base du portfolio avec Supabase

Revenez à ce cours lorsque votre base est en place.

III. Typer les lectures REST du portfolio

Partir des colonnes réellement créées

La table `articles` définit `.id` en `bigint`, `title` et `excerpt` en `text NOT NULL`.

L'API renvoie un tableau JSON, même pour une seule ligne filtrée.

```
import { z } from "zod";

const articleCardSchema = z.object({
  id: z.number().int().positive(),
  title: z.string().min(1),
  slug: z.string().min(1),
  excerpt: z.string(),
});

type ArticleCard = z.infer<typeof articleCardSchema>;

const payload: unknown = [{ id: 1, title: "TypeScript : les bases", slug: "typescript", excerpt: "Guide" }];

const articles: ArticleCard[] = z.array(articleCardSchema).parse(payload);

console.log(articles[0]?.title);
```

> *Remplacez `id: 1` par `id: "1"`. Pourquoi le typage statique seul ne suffit-il pas ? Schémas Zod · API REST Supabase*

Distinguer `null`, champ absent et champ non sélectionné

`cover_url`, `cover_alt`, `content` et `published_at` sont nullable dans le schema SQL.

Un champ omis par `select` est absent du JSON. Il ne vaut pas `null`.

```
import { z } from "zod";

const articleDetailSchema = z.object({
  id: z.number().int(),
  title: z.string(),
  slug: z.string(),
  excerpt: z.string(),
  content: z.string().nullable(),
  cover_url: z.string().nullable(),
  cover_alt: z.string().nullable(),
  published_at: z.iso.datetime({ offset: true }).nullable(),
});

const payload: unknown = {
  id: 1,
  title: "TypeScript et Zod",
  slug: "ts",
  excerpt: "Guide",
  content: null,
  cover_url: null,
  cover_alt: null,
  published_at: "2026-09-24T10:00:00+02:00"
};

console.log(articleDetailSchema.parse(payload).cover_url);
```

> *Retirez `cover_url` du payload. Que se passe-t-il ?*

Nullable et optional · Datetime ISO

Faire correspondre le schéma à `select`

Une projection REST ne contient que les colonnes demandées. Le schéma décrit cette projection.

```
import { z } from "zod";

const cardSchema = z.object({
  id: z.number().int(),
  title: z.string(),
  slug: z.string()
});

const cardsSchema = z.array(cardSchema);

const payload: unknown = [{ id: 1, title: "TypeScript", slug: "typescript" }];

console.log(cardsSchema.parse(payload), "\n");

const url = new URL("/rest/v1/articles", "https://example.supabase.co");
url.searchParams.set("select", "id,title,slug");
url.searchParams.set("order", "created_at.desc");
url.searchParams.set("limit", "5");
console.log(url.toString());
```

> *Ajoutez `excerpt` à `select` et au schéma, puis au payload de test.*

Sélection et tri PostgREST

Valider une liste reçue par `fetch`

```
import { z } from "zod";

const schema = z.array(z.object({
  id: z.number().int(),
  title: z.string(),
  slug: z.string()
}));

type ArticleCard = z.infer<typeof schema>[number];

async function fetchArticles(): Promise<ArticleCard[]> {

 const url = new URL("https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/articles");

  url.searchParams.set("select", "id,title,slug");

  const key = "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV";

  const response = await fetch(url, { headers: { apikey: key } });

  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const payload: unknown = await response.json();
  return schema.parse(payload);
}

console.log(await fetchArticles());
```

> *Que se passe-t-il si on retire `title` de `select`.*

Routes REST Supabase

Chercher un article par `slug`

Le filtre `slug=eq...` renvoie `[]` si le slug est absent ou si RLS masque le brouillon.

```ts
import { z } from "zod";
const schema = z.array(z.object({ id: z.number().int(), slug: z.string(), content: z.string().nullable() }));
async function articleBySlug(slug: string) {

  const url = new URL("https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/articles");
  url.searchParams.set("select", "id,slug,content");
  url.searchParams.set("slug", `eq.${slug}`);

  const apiKey = "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV";


  const response = await fetch(url, { headers: { apiKey } }); // Notation alternative

  if (!response.ok) throw new Error(`HTTP ${response.status}`);

  const payload: unknown = await response.json();

  return schema.parse(payload)[0] ?? null;
}

console.log(await articleBySlug("typescript-strict"));
```

> *Si un slug est inexistant, ou si on demande un article encore en brouillon. Quel est le résultat attendu ?*

Filtres PostgREST

Ne pas confondre tableau vide et erreur HTTP

Le `slug` est unique, mais la réponse par défaut reste un tableau de zéro ou une ligne.

```
import { z } from "zod";

const schema = z.array(
  z.object({
    id: z.number().int(),
    slug: z.string()
}));

function firstArticle(payload: unknown) {
  const rows = schema.parse(payload);
  if (rows.length > 1) throw new Error("Duplicate slug response");
  return rows[0] ?? null;
}

console.log(firstArticle([]));
console.log(firstArticle([{ id: 1, slug: "typescript-strict" }]));
```

> *Remplacer `rows[0] ?? null` par `rows[0]!` masque quel cas réel ?*

Réponses JSON PostgREST

Lire une page publiée

`pages.content` est obligatoire. Un slug `about` peut néanmoins renvoyer `[]` sous RLS.

```
import { z } from "zod";

const schema = z.array(
 z.object({
  id: z.number().int(),
  title: z.string(),
  slug: z.string(),
  content: z.string()
}));

async function fetchAboutPage() {

  const url = new URL("https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/pages");

  const apikey = "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV";

  url.searchParams.set("select", "id,title,slug,content");
  url.searchParams.set("slug", "eq.about");

  const response = await fetch(url, { headers: { apikey } });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const payload: unknown = await response.json();
  return schema.parse(payload)[0] ?? null;
}

console.log(await fetchAboutPage());
```

Défi : retirez `content` du `select`. Corrigez le schéma ou la requête.

API REST Supabase

Lire les projets avec leurs champs propres

`summary` et `description` sont obligatoires. `demo_url` et `repo_url` peuvent être `null`.

```
import { z } from "zod";

const projectSchema = z.object({
  id: z.number().int(),
  title: z.string(),
  slug: z.string(),
  summary: z.string(),
  description: z.string(),
  demo_url: z.string().nullable(),
  repo_url: z.string().nullable(),
});

const payload: unknown = [
  {
    id: 1,
    title: "Portfolio",
    slug: "portfolio",
    summary: "Vitrine",
    description: "Contenu",
    demo_url: null,
    repo_url: null
  }
];

console.log(z.array(projectSchema).parse(payload));
```

> `z.url().nullable()` *serait-il compatible avec une URL mal formée déjà stockée ?*

Schémas Zod · Script SQL du portfolio

Trier et limiter les projets publiés

Une liste de cartes n'a pas besoin de télécharger `description`.

```
import { z } from "zod";
const schema = z.array(
  z.object({
    id: z.number().int(),
    title: z.string(),
    slug: z.string(),
    summary: z.string(),
  })
);
async function fetchProjects() {
  const url = new URL("https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/projects");
  url.searchParams.set("select", "id,title,slug,summary");
  url.searchParams.set("order", "created_at.desc");
  url.searchParams.set("limit", "6");
  const response = await fetch(url, { headers: { apikey: "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV" } });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const payload: unknown = await response.json();
  return schema.parse(payload);
}
void fetchProjects().then(console.log).catch(console.error);
```

Défi : ajoutez le détail d'un projet filtré par `slug` sans changer le type de cette liste. Tri et pagination PostgREST

Lire les compétences

`skills.level` est nullable, `name` et `category` ne le sont pas. Les compétences n'ont pas de `published_at`.

```
import { z } from "zod";
const schema = z.array(
  z.object({
    id: z.number().int(),
    name: z.string(),
    category: z.string(),
    level: z.string().nullable(),
  })
);
async function fetchSkills() {
  const url = new URL("https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/skills");
  url.searchParams.set("select", "id,name,category,level");
  url.searchParams.set("order", "name.asc");
  const response = await fetch(url, { headers: { apikey: "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV" } });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const payload: unknown = await response.json();
  return schema.parse(payload);
}
void fetchSkills().then(console.log).catch(console.error);
```

Défi : remplacez `level: null` par un champ absent dans un test. Quel contrat est violé ? API REST Supabase

Filtrer les compétences par catégorie

Un filtre construit avec `URLSearchParams` encode la valeur fournie, sans concaténer une URL brute.

```
import { z } from "zod";
const schema = z.array(
  z.object({
    id: z.number().int(),
    name: z.string(),
    category: z.string(),
  })
);
async function skillsByCategory(category: string) {
  const url = new URL("https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/skills");
  url.searchParams.set("select", "id,name,category");
  url.searchParams.set("category", `eq.${category}`);
  const response = await fetch(url, { headers: { apikey: "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV" } });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const payload: unknown = await response.json();
  return schema.parse(payload);
}
void skillsByCategory("frontend").then(console.log).catch(console.error);
```

Défi : ajoutez `order=name.asc`. Quelle partie du schéma change ? Filtres PostgREST

Typer la table de liaison

`project_skills` ne contient que deux clés étrangères `bigint`, non nulles.

```
import { z } from "zod";
const linkSchema = z.object({
  project_id: z.number().int(),
  skill_id: z.number().int(),
});
const linksSchema = z.array(
  linkSchema
);
const payload: unknown = [{ project_id: 1, skill_id: 2 }];
console.log(linksSchema.parse(payload));
const url = new URL("https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/project_skills");
url.searchParams.set("select", "project_id,skill_id");
url.searchParams.set("project_id", "eq.1");
console.log(url.toString());
```

Défi : mettez `skill_id: "2"`. TypeScript peut-il vérifier ce JSON avant `parse` ? Relations PostgREST

Valider une relation imbriquée

La jointure du cours précédent sélectionne `skills(name)` depuis `projects`. `skills` est un tableau, car la relation passe par `project_skills`.

```
import { z } from "zod";
const projectSchema = z.object({
  title: z.string(),
  slug: z.string(),
  skills: z.array(
    z.object({
      name: z.string(),
    })
  ),
});
const payload: unknown = [{ title: "Portfolio", slug: "portfolio",
  skills: [{ name: "TypeScript" }, { name: "React" }] }];
console.log(z.array(projectSchema).parse(payload)[0]?.skills);
```

Piège : remplacez `skills` par un objet unique. Pourquoi le schéma refuse-t-il ? Jointures Supabase

Charger la relation depuis l'API

La projection imbriquée doit correspondre au schéma de validation.

```
import { z } from "zod";
const schema = z.array(
  z.object({
    title: z.string(),
    slug: z.string(),
    skills: z.array(
      z.object({
        name: z.string(),
      })
    ),
  })
);
async function projectWithSkills(slug: string) {
  const url = new URL("https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/projects");
  url.searchParams.set("select", "title,slug,skills(name)");
  url.searchParams.set("slug", `eq.${slug}`);
  const response = await fetch(url, { headers: { apikey: "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV" } });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const payload: unknown = await response.json();
  return schema.parse(payload)[0] ?? null;
}
void projectWithSkills("portfolio").then(console.log).catch(console.error);
```

Défi : ajoutez `category` dans `skills(...)`, puis mettez à jour le schéma imbriqué. Relations PostgREST

Paginer sans changer le type des lignes

`limit` et `offset` changent le nombre de lignes, pas leur type.

```
import { z } from "zod";
const schema = z.array(
  z.object({
    id: z.number().int(),
    title: z.string(),
  })
);
async function articlePage(page: number, size = 5) {
  if (!Number.isInteger(page) || page < 1) throw new Error("Invalid page");
  const url = new URL("https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/articles");
  url.searchParams.set("select", "id,title"); url.searchParams.set("order", "created_at.desc,id.desc");
  url.searchParams.set("limit", String(size)); url.searchParams.set("offset", String((page - 1) * size));
  const response = await fetch(url, { headers: { apikey: "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV" } });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const payload: unknown = await response.json();
  return schema.parse(payload);
}
void articlePage(2).then(console.log).catch(console.error);
```

Défi : calculez l'offset des pages 1, 2 et 3 pour `size = 5`. Pagination PostgREST

Vérifier une projection complète de projet

Le schéma complet ajoute les dates et les colonnes de couverture, absentes des cartes.

```
import { z } from "zod";
const date = z.iso.datetime({
  offset: true,
});
const schema = z.object({
  id: z.number().int(),
  created_at: date,
  published_at: date.nullable(),
  title: z.string(),
  slug: z.string(),
  summary: z.string(),
  description: z.string(),
  cover_url: z.string().nullable(),
  cover_alt: z.string().nullable(),
  demo_url: z.string().nullable(),
  repo_url: z.string().nullable(),
});
const payload: unknown = { id: 1, created_at: "2026-09-24T08:00:00Z", published_at: "2026-09-24T08:00:00Z", title: "Portfolio", slug: "
  summary: "Vitrine", description: "Contenu", cover_url: null,
  cover_alt: null, demo_url: null, repo_url: null };
console.log(schema.parse(payload).created_at);
```

Défi : retirez `published_at`. Pourquoi le schéma échoue-t-il ? Formats Zod · Script SQL

Respecter la lecture seule du portfolio

Le SQL du cours précédent accorde `SELECT` au rôle `anon`, mais retire `INSERT`, `UPDATE` et `DELETE`. RLS masque aussi les articles, pages et projets sans `published_at`.

```
type PublicMethod = "GET";
function publicRequest(path: string, method: PublicMethod = "GET") {
  const url = new URL(`/rest/v1/${path}`, "https://nxfqfvxpdsqsskobqent.supabase.co");
  return new Request(url, { method });
}
console.log(publicRequest("articles").method);
```

Piège : `publicRequest("articles", "POST")` doit être refusé par TypeScript. Les écritures demanderaient une autre politique et une vérification côté serveur. RLS Supabase

IV. Prouver le contrat

Séparer erreur HTTP et erreur de validation

Un `200` peut transporter des données incompatibles. `safeParse` permet de signaler le champ fautif.

```ts
import { z } from "zod";

const schema = z.array(
  z.object({
    id: z.number().int(),
    title: z.string(),
  })
);

function decodeArticles(payload: unknown) {
  const result = schema.safeParse(payload);
  if (result.success) return result.data;
  throw new Error(`Invalid articles response at ${result.error.issues[0]?.path.join(".") ?? "response"}`);
}

try {
  decodeArticles([{ id: 1, title: "OK" }, { id: 2, title: 42 }]);
} catch (error) {
  console.log("ERREUR !");
}
```

> *Quel est le résultat ? Pourquoi ne pas afficher le JSON complet ? Comment corriger ?*

Erreurs Zod

Tester ligne valide, champ invalide et liste vide

Placez ce bloc dans un fichier `*.test.ts` après installation de `vitest` et `zod`.

```
import { expect, it } from "vitest";
import { z } from "zod";
const schema = z.array(
  z.object({
    id: z.number().int(),
    title: z.string(),
  })
);
it("accepts an article", () => {
  expect(schema.safeParse([{ id: 1, title: "TS" }]).success).toBe(true);
});
it("rejects an invalid second row", () => {
  expect(schema.safeParse([{ id: 1, title: "TS" }, { id: 2, title: 42 }]).success).toBe(false);
});
it("accepts no published articles", () => {
  expect(schema.parse([])).toEqual([]);
});
```

Défi : ajoutez un test pour `content: null` sur le schéma de détail. Tests Vitest

TP : sécuriser les lectures du portfolio

> 🛠️ ***TP (~45 min)*** *: typer et valider les lectures Supabase avec Zod.*

Palier 1 obligatoire - Écrivez `decodeArticles(payload: unknown)` pour `id,title,slug,excerpt` et branchez-le sur `GET /rest/v1/articles?select=id,title,slug,excerpt` . Testez une ligne valide, un titre numérique et `[]` . Résultat attendu : seules les lignes conformes atteignent l'interface. `[]` affiche un état vide, sans erreur.

Palier 2 - Ajoutez une page `about` , les cartes de projets et les compétences. Définissez un schéma distinct pour chaque `select` , puis testez les champs nullable et un slug absent. Résultat attendu : les trois routes passent par Zod. Une réponse incomplète est refusée, un slug absent donne `null` .

Palier 3 bonus - Chargez `projects?select=title,slug,skills(name)` et paginez les articles. Affichez le chemin d'une erreur de validation sans divulguer le payload. Résultat attendu : compétences imbriquées typées, pages stables et erreur ciblée sur le champ fautif.

Ouvrir le TP · Correction

Documentation Zod · API REST Supabase